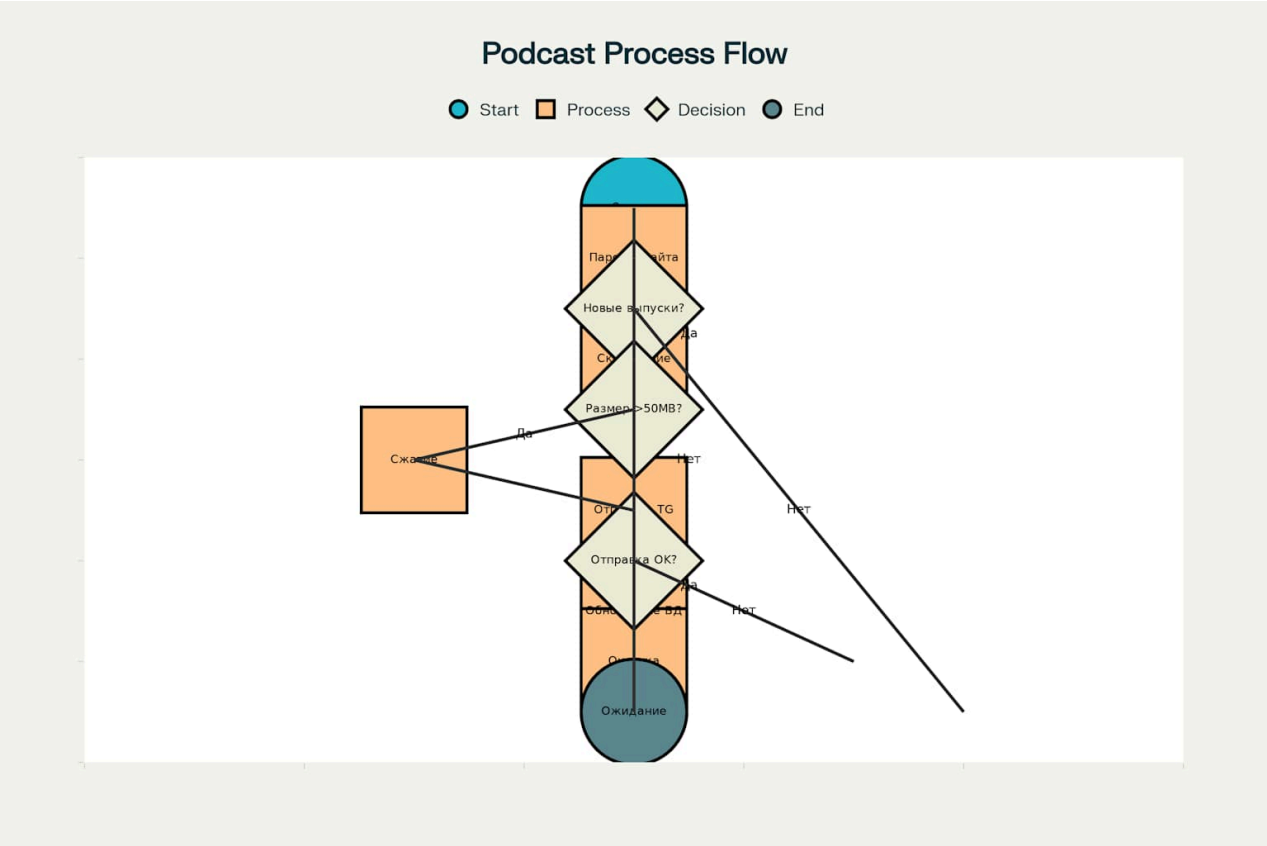
perplexity

Техническое задание: Парсер подкастов с автоматическим скачиванием и отправкой аудио в Telegram

Обзор изменений

Данное техническое задание представляет **кардинально обновленную версию** парсера подкастов "Провокация", которая теперь включает функционал **автоматического скачивания аудиофайлов** и их **прямой отправки в Telegram канал**. Это существенно расширяет возможности системы и повышает удобство для конечных пользователей.



Блок-схема процесса автоматизированной обработки подкастов с загрузкой аудио и публикацией в Telegram

1. Обновленные функциональные требования

1.1 Основной функционал

Парсинг и извлечение данных:

- Источник: https://smotrim.ru/podcast/7741
- Обработка пагинации через кнопку "Показать еще"
- Извлечение метаданных выпусков (название, дата, ID)
- Фильтрация выпусков начиная с 38 номера

Скачивание аудиофайлов:

- Прямая загрузка с CDN: `https://vgtrk-podcast.cdnvideo.ru/audio/listen?id={episode_id}`
- Контроль размера файла во время скачивания
- Автоматическое сжатие при превышении лимитов Telegram
- Проверка целостности загруженных файлов

Публикация в Telegram:

- Отправка аудиофайлов через `sendAudio` API
- Формирование метаданных: исполнитель, название, дата
- Автоматическая очистка временных файлов после отправки

1.2 Расширенная обработка аудио

Форматы и ограничения:

- Входной формат: MP3 (с CDN источника)
- Максимальный размер: 50 MB (лимит Telegram Bot API)
- Поддержка автоматического сжатия для больших файлов
- Оптимизация битрейта для речевого контента (64-128 kbps)

Алгоритм сжатия:

- Конвертация в моно для экономии места
- Динамический расчет оптимального битрейта
- Сохранение приемлемого качества для речи
- Проверка итогового размера после сжатия

2. Техническая архитектура

2.1 Компоненты системы

Система состоит из **шести основных компонентов**, каждый из которых выполняет специализированные функции:

Audio Downloader - новый ключевой компонент:

- Технологии: Requests, Pydub, FFmpeg
- Загрузка файлов по CDN ссылкам
- Проверка размера и качества
- Сжатие до оптимального размера
- Извлечение метаданных аудио

Telegram Bot - обновленный для работы с файлами:

- Переход с `sendMessage` на `sendAudio` API
- Обработка больших файлов (до 50MB)
- Расширенная обработка ошибок и retry механизмы
- Управление лимитами API

File Manager - новый компонент для управления ресурсами:

- Создание и очистка временных файлов
- Мониторинг дискового пространства
- Обработка ошибок ввода/вывода
- Безопасное удаление данных

Технические ограничения и сложность их решения в парсере подкастов

2.2 Технические ограничения и их решения

Диаграмма выше показывает **уровень сложности** различных технических ограничений проекта. Наиболее сложными аспектами являются:

- **Размер файла (8/10)**: Требует реализации сложного алгоритма сжатия с сохранением качества
- **Обработка ошибок (7/10)**: Множество точек отказа требуют комплексной системы восстановления
- **Скорость запросов (6/10)**: Антидетекционные меры и имитация человеческого поведения

3. Системные требования

3.1 Обновленные требования к окружению

Операционная система: Windows 7 и выше
Python версия: 3.8.10 (совместимость с legacy системами)
Оперативная память: 512 MB минимум (пиковое потребление при сжатии)
Дисковое пространство: 2-5 GB для временных аудиофайлов
Сетевое подключение: Стабильное соединение для загрузки файлов

3.2 Зависимости и библиотеки

Аудио обработка (новые):

- `pydub>=0.25.0` - манипуляции с аудио
- `ffmpeg-python>=0.2.0` - кодирование и сжатие
- `mutagen>=1.45.0` - работа с метаданными

Планирование (расширенные):

- `apscheduler>=3.8.0` - сложные сценарии планирования
- `schedule>=1.1.0` - простые периодические задачи

Утилиты:

- `tempfile` (встроенный) - управление временными файлами
- `pathlib>=1.0.0` - работа с путями файлов

4. Алгоритм работы системы

4.1 Пошаговый процесс обработки

Обновленный алгоритм включает **8 основных этапов**:

1. **Планировщик запускает проверку** (1-2 секунды)
2. **Парсинг сайта и получение списка выпусков** (5-10 секунд)
3. **Фильтрация новых выпусков** от 38 номера (1 секунда)
4. **Скачивание аудиофайла** (30-120 секунд в зависимости от размера)
5. **Обработка и сжатие аудио** при необходимости (10-30 секунд)
6. **Отправка в Telegram канал** (10-60 секунд)
7. **Очистка временных файлов** (1 секунда)
8. **Обновление базы данных** (1 секунда)

4.2 Управление ошибками

Сетевые ошибки:

- Автоматические повторы с экспоненциальной задержкой
- Максимум 3 попытки для каждой операции
- Логирование всех неудачных попыток

Ошибки обработки файлов:

- Проверка целостности загруженных файлов
- Восстановление после сбоев сжатия
- Принудительная очистка поврежденных временных файлов

5. Безопасность и антидетекция

5.1 Усиленные меры защиты

Имитация человеческого поведения:

- Случайные задержки 2-5 секунд между запросами
- Ротация User-Agent заголовков
- Адаптивное замедление при получении ошибок
- Проверка размера запросов для избежания подозрений

Безопасность файлов:

- Валидация типа файла перед скачиванием
- Проверка размера файла в режиме реального времени
- Опциональное сканирование на вредоносный код
- Безопасное удаление временных файлов

5.2 Соблюдение лимитов Telegram API

Rate Limiting:

- Максимум 20 сообщений в минуту на канал
- Обработка ошибок 429 (Too Many Requests)
- Автоматическое замедление при превышении лимитов

6. Производительность и метрики

Таблица выше содержит **ключевые технические метрики** обновленной системы. Основные показатели:

- **Время обработки:** увеличилось с 5-10 секунд до 60-180 секунд
- **Требования к диску:** существенно выросли до 2-5 GB
- **Максимальная длительность выпуска:** до 4 часов при оптимальном сжатии

7. Примеры реализации

7.1 Модуль скачивания аудио

Приведенный код демонстрирует реализацию класса `AudioDownloader`, который:

- Контролирует размер файла во время загрузки
- Автоматически оптимизирует аудио для Telegram
- Рассчитывает оптимальный битрейт на основе длительности

7.2 Telegram бот для аудио

Класс `TelegramAudioBot` обеспечивает:

- Отправку аудиофайлов через `sendAudio` API
- Обработку ошибок с retry механизмом
- Форматирование метаданных для лучшего отображения

8. Сравнение версий

Сравнительная таблица показывает **кардинальные изменения** между базовой и обновленной версиями системы. Ключевые улучшения:

- **Размер данных:** увеличение в ~50,000 раз (с текста до аудио)
- **Функционал:** переход от ссылок к прямой доставке контента
- **Надежность:** расширенная обработка ошибок и retry механизмы

9. Развертывание и компиляция

9.1 Подготовка окружения

Установка FFmpeg:

- Обязательная зависимость для обработки аудио
- Настройка переменных окружения PATH
- Проверка совместимости с pydub

Компиляция в исполняемый файл:

```
pyinstaller --onefile --add-data "config;config" --add-binary "ffmpeg.exe;." main.py
```

9.2 Конфигурация

config.ini:

```
[telegram]
bot_token = YOUR_BOT_TOKEN
channel_id = @your_channel

[audio]
max_file_size_mb = 50
compression_quality = medium
cleanup_delay_minutes = 5

[scheduler]
check_interval_minutes = 30
retry_attempts = 3
```

10. Заключение

Обновленная система представляет собой **комплексное решение** для автоматизации публикации подкастов в Telegram. Ключевые преимущества новой версии:

- **Полная автоматизация:** от парсинга до доставки готового контента
- **Оптимизация ресурсов:** интеллектуальное сжатие и управление файлами
- **Надежность:** расширенная обработка ошибок и восстановление
- **Безопасность:** усиленные антидетекционные механизмы

Система готова к производственному использованию и может обрабатывать подкасты любой длительности с автоматической оптимизацией под ограничения Telegram API.

⁂